

TAREA ACADÉMICA 1

Aplicación de DevOps y pipeline CI/CD

Caso: AgroSmart Monitor

Curso	Herramientas de Desarrollo Profesional - TIC
Estudiante(s)	[Nombres y apellidos de los integrantes del Equipo]
Sección	[Número de sección]
Docente	[Nombre del docente]
Repositorio	github.com/<usuario>/agrosmart-monitor
Fecha	Agosto de 2026

Resumen ejecutivo

AgroSmart Monitor es un proyecto demostrativo desarrollado en Java 17 y Maven para simular el monitoreo de temperatura y humedad de una cámara de frío agrícola. El proyecto se alojó en GitHub y se configuró un workflow de GitHub Actions que automatiza las pruebas, la construcción, el despliegue simulado y la publicación del artefacto. Esta TA1 demuestra cómo las prácticas DevOps conectan el cambio de código con una validación automática y una retroalimentación verificable mediante los logs de ejecución.

Resultado principal: el pipeline detectó una prueba inconsistente, detuvo las etapas posteriores y mostró en los logs la causa exacta: **expected NORMAL but was ALERT**. Tras corregir la expectativa de la prueba, la ejecución #8 completó satisfactoriamente las pruebas, el build, el despliegue simulado y la publicación del artefacto.

1. Objetivos de la TA1

- Implementar un pipeline de CI/CD en un proyecto alojado en GitHub.
- Aplicar prácticas iniciales de DevOps: versionamiento, automatización, pruebas y retroalimentación continua.
- Interpretar y comparar la trazabilidad de una ejecución fallida y de una ejecución exitosa mediante los logs.
- Explicar el impacto de una prueba fallida sobre la compilación, el despliegue y la generación del artefacto.

2. Alcance

La implementación corresponde a una aplicación didáctica inicial de DevOps. Incluye control de versiones, integración continua, pruebas automatizadas, empaquetado, despliegue simulado y revisión de logs. No incluye todavía contenedorización, infraestructura como código, análisis automático de seguridad, monitoreo de producción ni despliegue real.

3. Descripción del proyecto

El sistema simula lecturas de temperatura y humedad. Cada temperatura se clasifica en uno de tres estados: NORMAL hasta 8 °C, WARNING por encima de 8 °C y hasta 10 °C, y ALERT por encima de 10 °C. Cuando se obtiene ALERT, el sistema genera automáticamente un reporte de texto.

Arquitectura funcional de AgroSmart Monitor

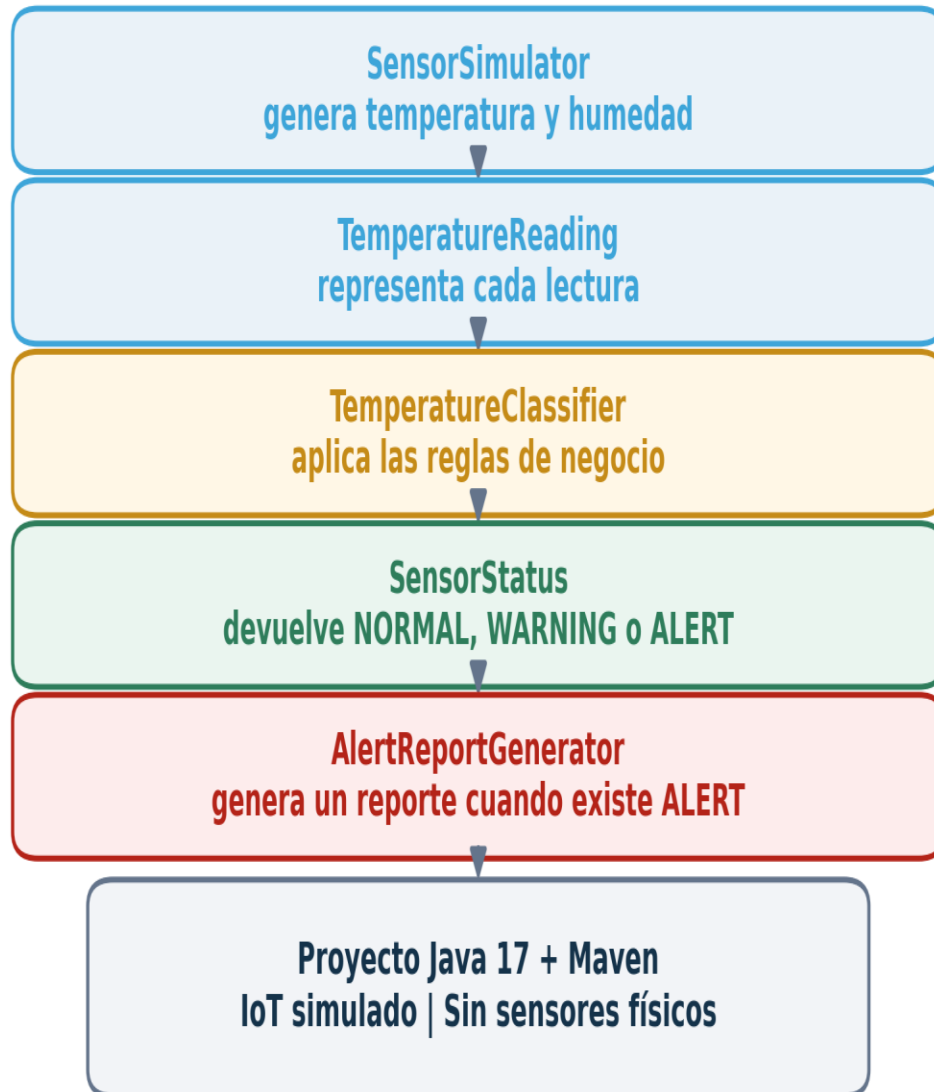


Figura 1. Arquitectura funcional simplificada de AgroSmart Monitor.

3.1 Organización del repositorio

```

agrosmart-monitor/
├── .github/workflows/ci.yml    # Workflow de GitHub Actions
├── pom.xml                    # Configuración de Maven
├── README.md                  # Descripción y forma de ejecución
├── src/main/java/com/agrosmart/ # Código de la aplicación
└── src/test/java/com/agrosmart/ # Pruebas automatizadas con JUnit

```

3.2 Herramientas empleadas

Herramienta	Uso en el proyecto	Evidencia
Git y GitHub	Versionamiento y alojamiento del código.	Repositorio e historial de commits.
GitHub Actions	Automatización del workflow CI/CD.	Ejecuciones, jobs, pasos y logs.
Java 17	Lenguaje y entorno de ejecución.	Código fuente y configuración del runner.
Maven	Pruebas, compilación y empaquetado.	pom.xml y comandos mvn.
JUnit 5	Pruebas automatizadas de la lógica.	TemperatureClassifierTest y SensorSimulatorTest.

4. Aplicación de DevOps

DevOps (Development and Operations - Desarrollo y Operaciones) se aplicó como un flujo de trabajo que integra la modificación del software con su validación, construcción, entrega simulada y análisis de resultados. No se limitó al uso de una herramienta: cada práctica produjo una evidencia verificable.

Ciclo DevOps aplicado al proyecto

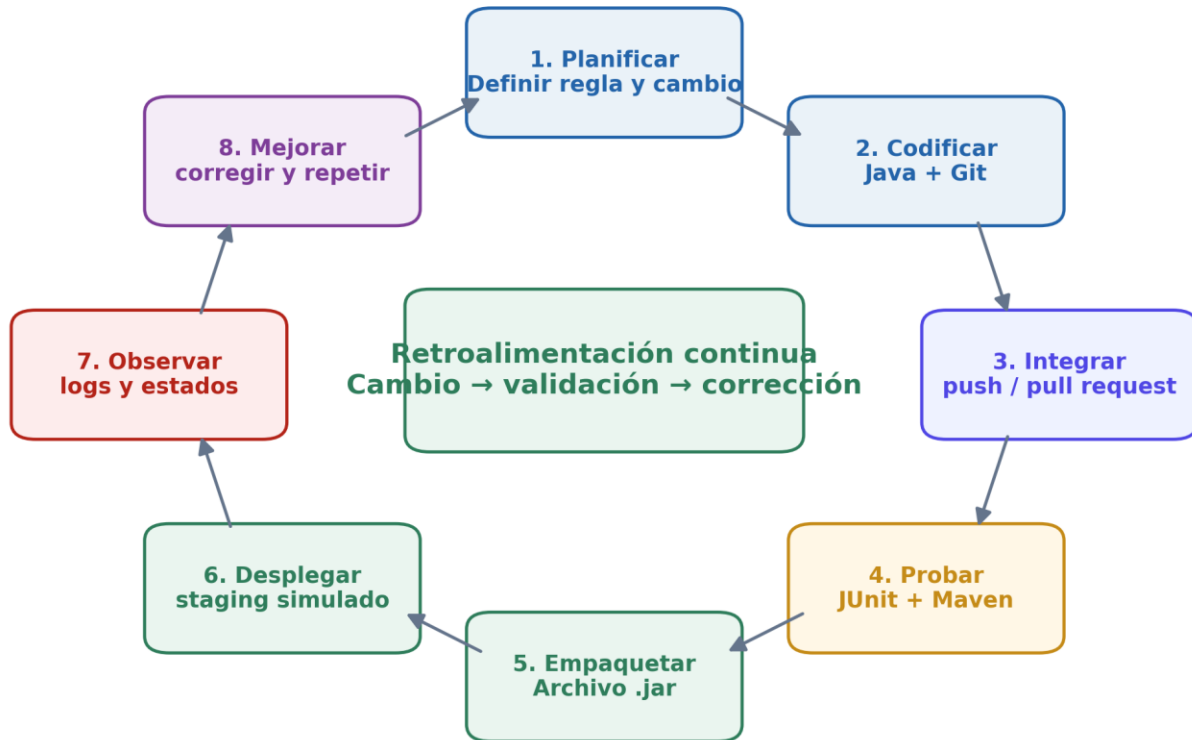


Figura 2. Ciclo DevOps aplicado al proyecto AgroSmart Monitor.

Interpretación: el valor DevOps está en reducir el tiempo entre un cambio y la detección del problema. El equipo recibe retroalimentación automática antes de desplegar un artefacto incorrecto.

4.1 Prácticas y evidencias verificables

Práctica DevOps	Aplicación realizada	Evidencia verificable
Colaboración	Repositorio compartido para centralizar el trabajo del equipo.	Colaboradores y commits.
Control de versiones	Registro de cambios mediante commits y push a main.	Historial del repositorio.
Integración continua	Cada push o pull request activa el workflow.	Workflow run en Actions.
Pruebas automáticas	Maven ejecuta las pruebas JUnit antes del build.	Logs del paso mvn -B test.
Entrega automatizada	Se empaqueta el .jar, se copia a staging y se publica como artefacto.	Pasos build, deploy y upload-artifact.
Retroalimentación	Un fallo impide continuar y señala la causa exacta.	BUILD FAILURE y mensaje de aserción.
Mejora continua	El equipo corrige el cambio y vuelve a ejecutar el pipeline.	Commit correctivo y nueva ejecución.

5. Implementación del pipeline CI/CD

CI/CD (Continuous Integration and Continuous Delivery/Deployment - Integración continua y entrega/despliegue continuos) se implementó en el archivo `.github/workflows/ci.yml`. El workflow se activa con cambios sobre la rama main y contiene un job denominado build-test-deploy.

Organización del workflow CI/CD

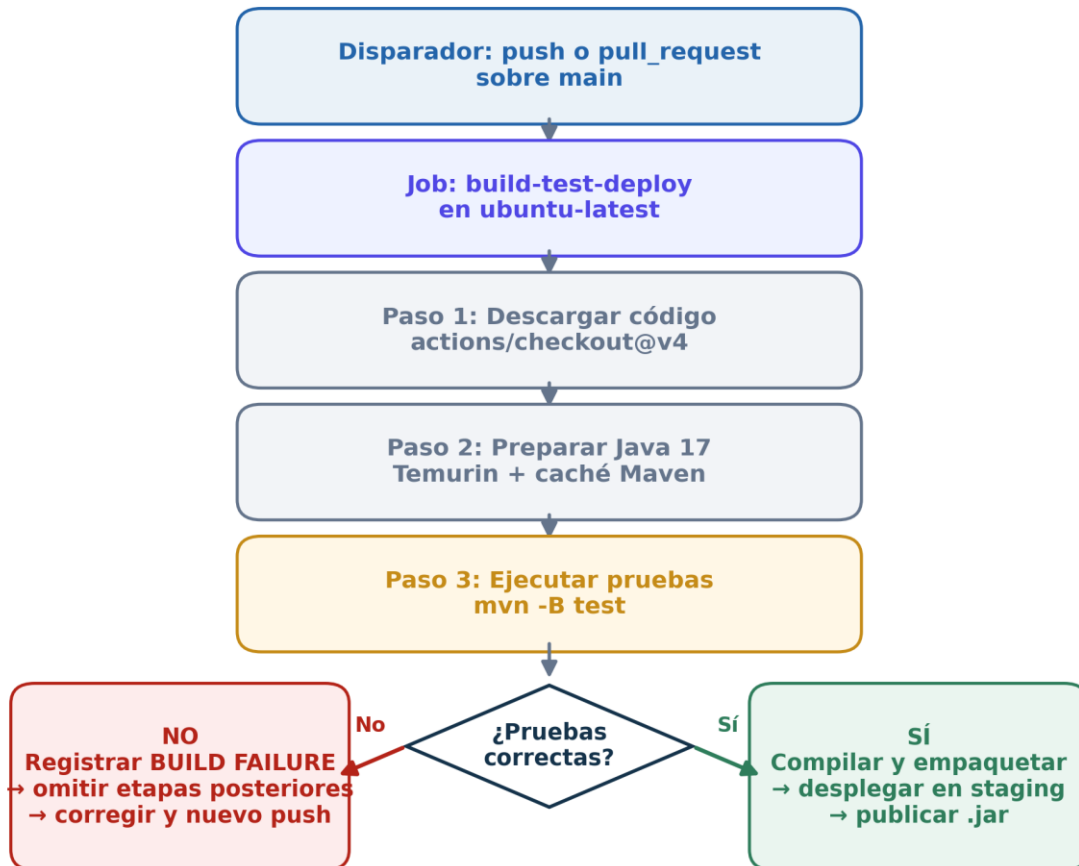


Figura 3. Flujo de control del pipeline y comportamiento ante una prueba fallida.

5.1 Estructura lógica

Nivel	Nombre	Función
Workflow	CI Pipeline - AgroSmart Monitor	Define el proceso automatizado completo.
Disparador	push / pull_request	Inicia una ejecución cuando cambia main.
Job	build-test-deploy	Agrupar los pasos ejecutados en ubuntu-latest.
Step	Ejecutar pruebas automatizadas	Ejecuta mvn -B test y detiene el job si falla.
Step	Compilar y empaquetar	Genera el archivo .jar si las pruebas pasan.
Step	Desplegar (simulado)	Copia el .jar a la carpeta staging.
Step	Publicar artefacto	Adjunta el .jar a la ejecución.

5.2 Configuración principal

```

name: CI Pipeline - AgroSmart Monitor

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

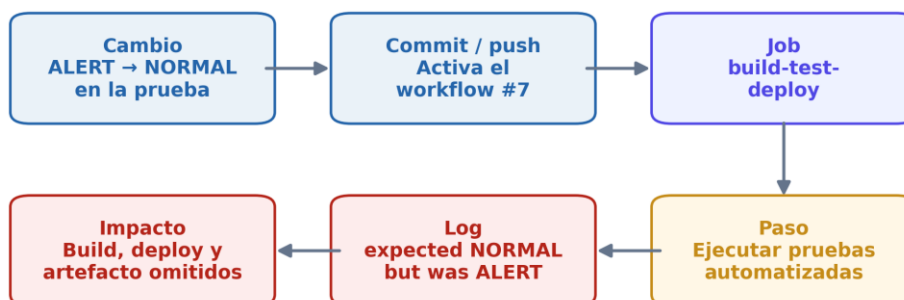
jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
          cache: 'maven'
      - name: Ejecutar pruebas automatizadas
        run: mvn -B test
      - name: Compilar y empaquetar
        run: mvn -B package -DskipTests

```


6. Trazabilidad de la ejecución

La trazabilidad permite relacionar el cambio realizado con la ejecución activada, el job, el paso que falló, el mensaje técnico y las etapas que quedaron bloqueadas. Los textos desplegados dentro de cada paso son los logs o registros de ejecución.

Cadena de trazabilidad de la ejecución fallida



La trazabilidad permite ubicar qué cambió, qué ejecución se activó, dónde falló y qué etapas quedaron bloqueadas.

Figura 4. Cadena de trazabilidad identificada en la ejecución número 7.

6.1 Escenario de prueba controlada

La regla de negocio clasifica una temperatura superior a 10 °C como ALERT. Para demostrar la capacidad de detección del pipeline, se modificó deliberadamente la expectativa de una prueba para que esperara NORMAL.

```
// Prueba modificada deliberadamente para provocar el fallo
assertEquals(SensorStatus.NORMAL,
    TemperatureClassifier.classify(10.1));

// Resultado real de la aplicación
TemperatureClassifier.classify(10.1) → SensorStatus.ALERT
```

Hipótesis de validación: si las pruebas protegen correctamente la regla de negocio, el pipeline debe marcar la ejecución como fallida y no debe continuar con el build ni con el despliegue.

7. Evidencias de GitHub Actions

Las siguientes capturas corresponden a la ejecución denominada “Demostrar detección de una prueba fallida #7”. Se presenta cada evidencia junto con su interpretación técnica.

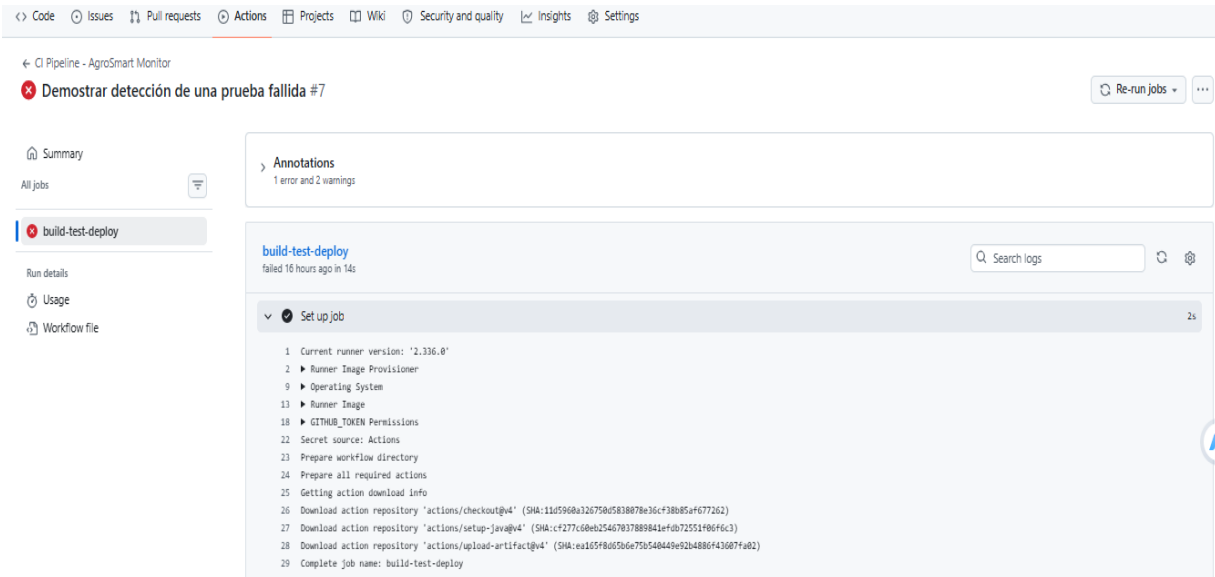


Figura 5. Resumen de la ejecución fallida y selección del job build-test-deploy.

Lectura de la evidencia: la ejecución terminó en estado failed después de 14 segundos. GitHub reportó una anotación de error y dos advertencias. El elemento build-test-deploy es el job; no es el workflow completo.

7.1 Preparación del entorno

The screenshot shows a GitHub Actions workflow log for a job named 'build-test-deploy'. The log is divided into three main steps:

- Descargar el código (checkout)**: Completed successfully in 1s.
- Preparar Java 17**: Completed successfully in 1s. This step includes:
 - Warning: Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the `ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true` environment variable.
 - Run `actions/setup-java@v4`.
 - Warning: `setup-java v4` is deprecated and will no longer receive updates. Please migrate to `actions/setup-java@v5`.
 - Installed distributions.
 - Creating settings.xml with server-id: github.
 - Writing to `/home/runner/.m2/settings.xml`.
 - Cache hit for: `setup-java-linux-x64-maven-7b144edc313d41f57a92a9bb63531ae1a568c5401008d46f8a5db079c06d798`.
 - (node:2241) [DEP0169] DeprecationWarning: `'url.parse()'` behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for `'url.parse()'` vulnerabilities.
 - Received 17152474 of 17152474 (100.0%), 16.9 MBs/sec.
 - Cache Size: ~16 MB (17152474 B).
 - Run `/usr/bin/tar -xf /home/runner/work/_temp/bed901df-60ac-4dcb-a860-cf3beb370aee/cache.tzst -P -C /home/runner/work/agrosmart-monitor/agrosmart-monitor --use-compress-program unxzstd`.
 - Cache restored successfully.
 - Cache restored from key: `setup-java-linux-x64-maven-7b144edc313d41f57a92a9bb63531ae1a568c5401008d46f8a5db079c06d798`.
- Ejecutar pruebas automatizadas**: Started, with a duration of 6s. The log shows:
 - Run `mvn -B test`.
 - [INFO] Scanning for projects...
 - [INFO]
 - [INFO] -----< com.agrosmart:agrosmart-monitor >-----
 - [INFO] Building AgroSmart Monitor 0.1.0
 - [INFO] from pom.xml
 - [INFO] -----[jar]-----
 - [INFO]
 - [INFO] --- resources:3.4.0:resources (default-resources) @ agrosmart-monitor ---
 - [INFO] skip non existing resourceDirectory `/home/runner/work/agrosmart-monitor/agrosmart-monitor/src/main/resources`
 - [INFO]
 - [INFO] --- compiler:3.15.0:compile (default-compile) @ agrosmart-monitor ---
 - [INFO] Recompiling the module because of changed source code.
 - [INFO] Compiling 6 source files with `Javac` compiler 17.0 to `target/classes`

Figura 6. Preparación de Java 17 e inicio del paso de pruebas automatizadas.

Lectura de la evidencia: la descarga del repositorio y la preparación de Java 17 finalizaron correctamente. El fallo no pertenece a la configuración inicial del runner; ocurre después, en la ejecución de las pruebas.

7.2 Identificación de la causa

```

36 [ERROR] com.agrosmart.TemperatureClassifierTest.debeClasificarComoAlertaSobre10Grados -- Time elapsed: 0.006 s <<< FAILURE!
37 org.opentest4j.AssertionFailedError: expected: <NORMAL> but was: <ALERT>
38     at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
39     at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
40     at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
41     at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:182)
42     at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:177)
43     at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:1145)
44     at com.agrosmart.TemperatureClassifierTest.debeClasificarComoAlertaSobre10Grados(TemperatureClassifierTest.java:29)
45     at java.base/java.lang.reflect.Method.invoke(Method.java:569)
46     at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
47     at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)
48
49 [INFO] Running com.agrosmart.SensorSimulatorTest
50 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in com.agrosmart.SensorSimulatorTest
51 [INFO]
52 [INFO] Results:
53 [INFO]
54 [ERROR] Failures:
55 [ERROR]   TemperatureClassifierTest.debeClasificarComoAlertaSobre10Grados:29 expected: <NORMAL> but was: <ALERT>
56 [INFO]
57 [ERROR] Tests run: 4, Failures: 1, Errors: 0, Skipped: 0
58 [INFO]
59 [INFO] -----
60 [INFO] BUILD FAILURE
61 [INFO] -----
62 [INFO] Total time: 3.014 s
63 [INFO] Finished at: 2026-08-30T02:08:23Z
64 [INFO] -----
65 [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test (default-test) on project agrosmart-monitor: There are test failures.
66 [ERROR]
67 [ERROR] Please refer to /home/runner/work/agrosmart-monitor/agrosmart-monitor/target/surefire-reports for the individual test results.
68 [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
69 [ERROR] -> [Help 1]
70 [ERROR]

```

Figura 7. Log de Maven con el resultado de las pruebas y la asección fallida.

Diagnóstico: Maven ejecutó cuatro pruebas: tres fueron satisfactorias y una falló. El mensaje indica expected: NORMAL but was: ALERT en TemperatureClassifierTest.java, línea 29. La infraestructura funcionó; la causa fue una expectativa de prueba inconsistente con la regla de negocio.

Dato observado	Valor	Interpretación
Tests run	4	Se ejecutaron cuatro pruebas.
Failures	1	Una asección no coincidió.
Errors	0	No hubo error inesperado del entorno.
Skipped	0	Ninguna prueba fue omitida.
Resultado	BUILD FAILURE	Maven devolvió una ejecución fallida.

7.3 Impacto sobre las etapas posteriores

build-test-deploy
failed 16 hours ago in 14s

Search logs

❌ Ejecutar pruebas automatizadas 6s

```

56 [INFO]
57 [ERROR] Tests run: 4, Failures: 1, Errors: 0, Skipped: 0
58 [INFO]
59 [INFO] -----
60 [INFO] BUILD FAILURE
61 [INFO] -----
62 [INFO] Total time: 3.014 s
63 [INFO] Finished at: 2026-08-30T02:08:23Z
64 [INFO] -----
65 [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.2.5:test (default-test) on project agrosmart-monitor: There are test failures.
66 [ERROR]
67 [ERROR] Please refer to /home/runner/work/agrosmart-monitor/agrosmart-monitor/target/surefire-reports for the individual test results.
68 [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
69 [ERROR] -> [Help 1]
70 [ERROR]
71 [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
72 [ERROR] Re-run Maven using the -X switch to enable full debug logging.
73 [ERROR]
74 [ERROR] For more information about the errors and possible solutions, please read the following articles:
75 [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
76 Error: Process completed with exit code 1.

```

- ⌚ Compilar y empaquetar (build) 0s
- ⌚ Desplegar (simulado) 0s
- ⌚ Publicar el .jar generado como artefacto de esta ejecución 0s
- > ✔ Post Preparar Java 17 1s
- > ✔ Post Descargar el código (checkout) 0s
- > ✔ Complete job 0s

Figura 8. Código de salida 1 y etapas posteriores omitidas.

Efecto de control: el proceso terminó con exit code 1. Por ello, GitHub Actions omitió Compilar y empaquetar, Desplegar (simulado) y Publicar el .jar. Este comportamiento evita entregar un artefacto que no superó las pruebas.

7.4 Cierre automático del job

The screenshot shows the log for a workflow named 'build-test-deploy' which failed 16 hours ago. The log displays the following steps:

- Publicar el jar generado como artefacto de esta ejecución** (0s)
- Post Preparar Java 17** (1s)
 - Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the `ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true` environment variable. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>
 - Post job cleanup.
 - (node:2364) [DEP0040] DeprecationWarning: The 'punycode' module is deprecated. Please use a userland alternative instead.
 - (Use 'node --trace-deprecation ...' to show where the warning was created)
- Post Descargar el código (checkout)** (0s)
 - Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the `ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true` environment variable. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>
 - Post job cleanup.
 - `/usr/bin/git version`
 - `git version 2.55.0`
 - Temporarily overriding HOME='/home/runner/work/_temp/9f71ef58-c433-4c02-9b5e-bd312577a7b9' before making global git config changes
 - Adding repository directory to the temporary git global config as a safe directory
 - `/usr/bin/git config --global --add safe.directory /home/runner/work/agrosmart-monitor/agrosmart-monitor`
 - `/usr/bin/git config --local --name-only --get-regexp core.sshCommand`
 - `/usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"`
 - `/usr/bin/git config --local --name-only --get-regexp http.https://github.com/.extraheader`
 - `http.https://github.com/.extraheader`
 - `/usr/bin/git config --local --unset-all http.https://github.com/.extraheader`
 - `/usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"`
 - `/usr/bin/git config --local --name-only --get-regexp includeIf.gitdir:`
 - `/usr/bin/git submodule foreach --recursive git config --local --show-origin --name-only --get-regexp remote.origin.url`
- Complete job** (0s)
 - Cleaning up orphan processes
 - Warning:** Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-java@v4. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

Figura 9. Ejecución de pasos de limpieza y cierre después del fallo.

Los pasos identificados como Post Preparar Java 17, Post Descargar el código y Complete job corresponden a tareas automáticas de limpieza. Su ejecución no significa que el pipeline haya sido satisfactorio; el estado general continúa siendo failed.

8. Corrección y nueva validación

La prueba debe representar la regla definida para el sistema. La corrección consiste en restituir ALERT como resultado esperado para una temperatura de 10.1 °C.

```
// Prueba corregida
assertEquals(SensorStatus.ALERT,
    TemperatureClassifier.classify(10.1));
```

8.1 Secuencia de recuperación

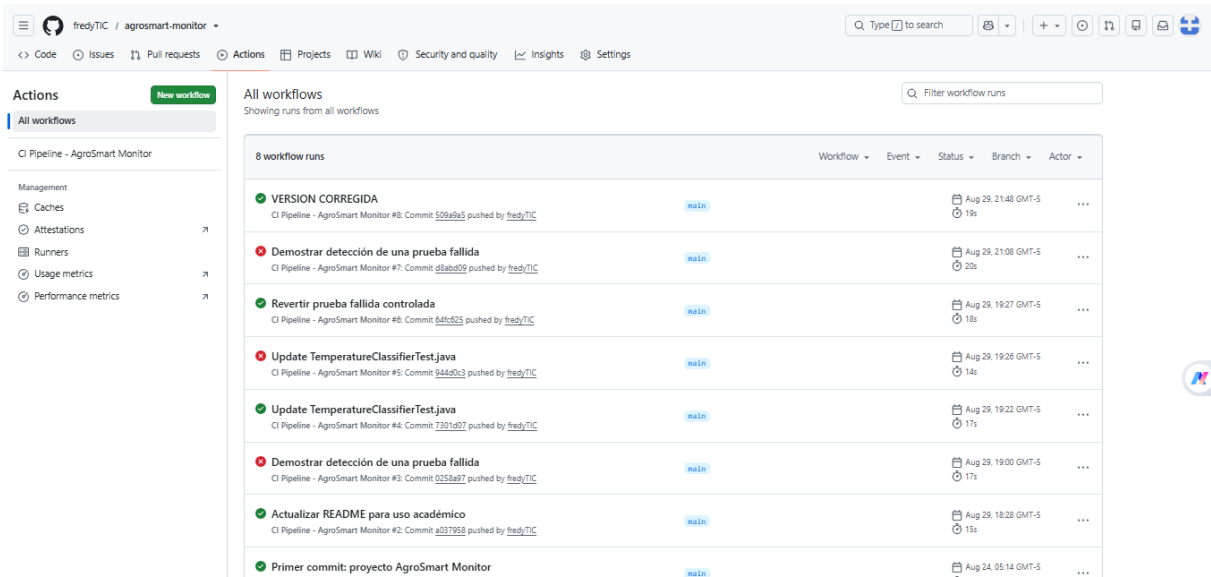
1. Modificar TemperatureClassifierTest.java y restituir SensorStatus.ALERT.
2. Crear un commit con un mensaje descriptivo, por ejemplo: Corregir resultado esperado para temperatura mayor a 10 °C.
3. Realizar push a la rama main.
4. Verificar que GitHub Actions inicie una nueva ejecución.
5. Confirmar que las pruebas, el build, el despliegue simulado y la publicación del artefacto terminen correctamente.
6. Adjuntar una captura de la nueva ejecución exitosa como evidencia final del equipo.

Evidencia confirmada: la ejecución VERSION CORREGIDA #8 finalizó en Success. Las cuatro pruebas pasaron y el pipeline continuó con la compilación, el despliegue simulado y la publicación del archivo .jar como artefacto.

8.2 Evidencias de la ejecución exitosa

Las siguientes capturas corresponden a la ejecución VERSION CORREGIDA #8, **activada por el commit 509a9a5**. Se muestran el resultado general, los pasos ejecutados y los logs que prueban que el pipeline completó el flujo previsto.

8.2.1 Resultado general y secuencia del job



The screenshot shows the GitHub Actions interface for the 'CI Pipeline - AgroSmart Monitor' workflow. The 'All workflows' tab is selected, displaying a list of 8 workflow runs. The runs are filtered by 'main' branch. The table below summarizes the visible runs:

Workflow	Event	Status	Branch	Actor
VERSION CORREGIDA	CI Pipeline - AgroSmart Monitor #8: Commit 509a9a5 pushed by fredyTIC	Completed	main	fredyTIC
Actualizar README para uso académico	CI Pipeline - AgroSmart Monitor #2: Commit a027956 pushed by fredyTIC	Completed	main	fredyTIC
Primer commit: proyecto AgroSmart Monitor		Completed	main	

Figura 10. Historial de ejecuciones que relaciona el fallo, la corrección y la nueva validación.

The screenshot displays the GitHub Actions interface for the repository 'fredyTIC / agrosmart-monitor'. The selected workflow is 'VERSION CORREGIDA #8', which is in a 'Success' state. The summary shows it was triggered by a push 17 hours ago, completed in 19 seconds, and produced one artifact. The job 'build-test-deploy' is shown as successful with a duration of 14s. The workflow file is 'ci.yml'. Annotations include two warnings about deprecated Node.js versions (20 and 14) used in the 'build-test-deploy' job.

fredyTIC / agrosmart-monitor

CI Pipeline - AgroSmart Monitor

VERSION CORREGIDA #8

Summary

All jobs

build-test-deploy

Run details

Usage

Workflow file

Triggered via push 17 hours ago

fredyTIC pushed [509a2a5](#) [main](#)

Status: Success

Total duration: 19s

Artifacts: 1

ci.yml

on: push

build-test-deploy 14s

Annotations

2 warnings

build-test-deploy

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-java@v4, actions/upload-artifact@4. For more information s...

Show more

build-test-deploy

setup-java v4 is deprecated and will no longer receive updates. Please migrate to actions/setup-java@v5.

Figura 11. Resumen de la ejecución #8: estado Success y un artefacto publicado.

Lectura de la evidencia: la ejecución correctiva aparece inmediatamente después de la ejecución fallida. El resumen registra Success, una duración total de 19 segundos y un artefacto, por lo que existe continuidad verificable entre cambio, nueva ejecución y resultado.

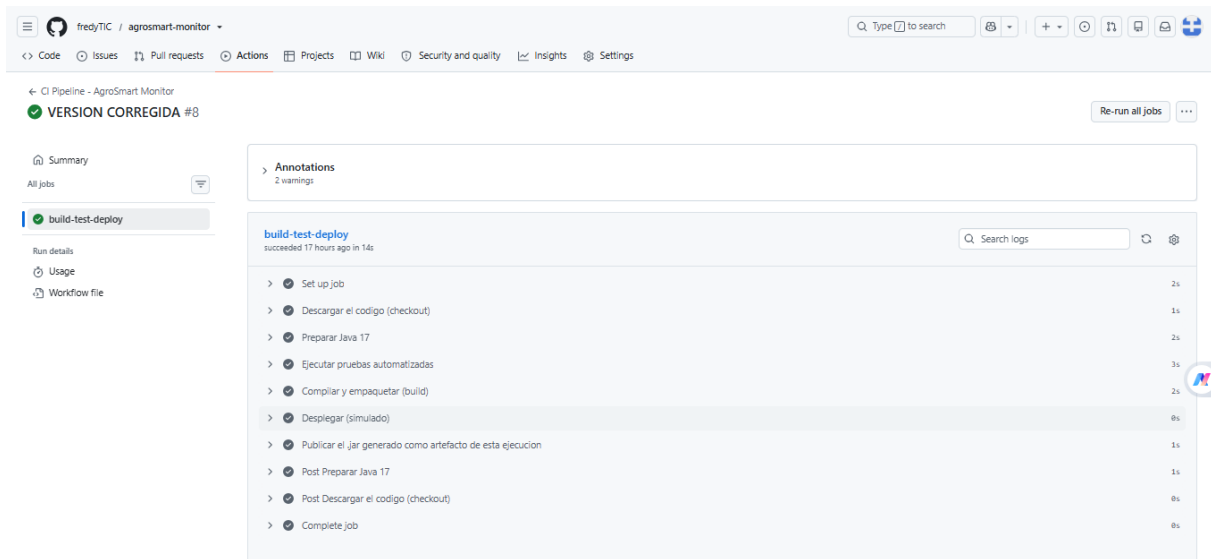


Figura 12. Secuencia completa del job build-test-deploy ejecutada satisfactoriamente.

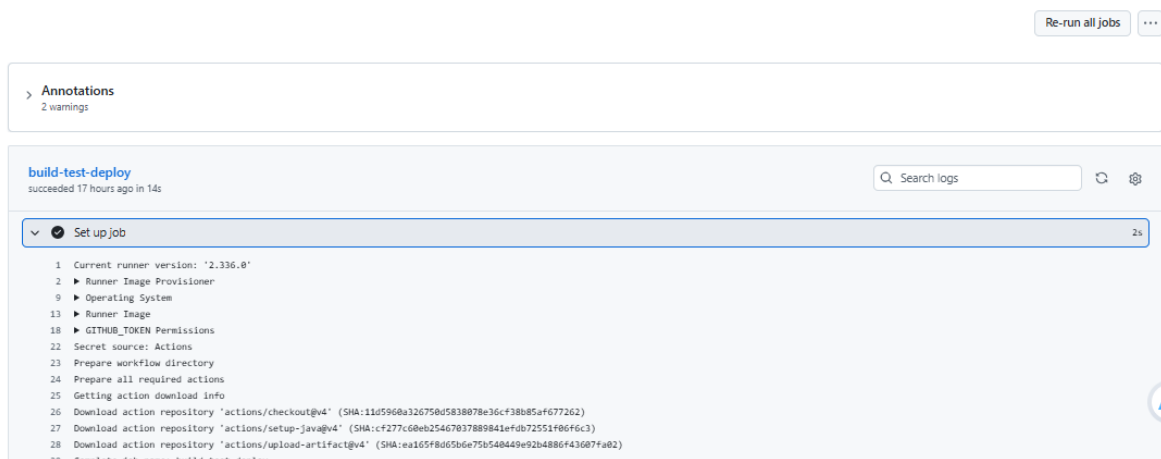


Figura 13. Preparación del runner e identificación de las acciones requeridas.

Interpretación: el job build-test-deploy duró 14 segundos y todos sus pasos muestran una marca satisfactoria. Set up job identifica el runner temporal y descarga las acciones antes de ejecutar el código del proyecto.

8.2.2 Descarga del código y preparación de Java

The screenshot shows the 'build-test-deploy' workflow logs. The 'Descargar el código (checkout)' step is expanded, showing the following commands and output:

```

1 Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true environment variable. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
2 ▶ Run actions/checkout@v4
18 Syncing repository: fredyTIC/agrosmart-monitor
19 ▶ Getting Git version info
23 Temporarily overriding HOME='/home/runner/work/_temp/fada5576-6e14-40fc-a4aa-e779587f63d6' before making global git config changes
24 Adding repository directory to the temporary git global config as a safe directory
25 /usr/bin/git config --global --add safe.directory /home/runner/work/agrosmart-monitor/agrosmart-monitor
26 Deleting the contents of '/home/runner/work/agrosmart-monitor/agrosmart-monitor'
27 ▶ Initializing the repository
44 ▶ Disabling automatic garbage collection
46 ▶ Setting up auth
54 ▶ Fetching the repository
58 ▶ Determining the checkout info
59 /usr/bin/git sparse-checkout disable
60 /usr/bin/git config --local --unset-all extensions.worktreeConfig
61 ▶ Checking out the ref
65 /usr/bin/git log -1 --format=%H
66 509a9a5c053678fe842cd98094fa8e7969c12a3
  
```

Figura 14. Checkout del repositorio y recuperación del commit correctivo.

The screenshot shows the 'build-test-deploy' workflow logs. The 'Preparar Java 17' step is expanded, showing the following commands and output:

```

1 Node 20 is being deprecated. This workflow is running with Node 24 by default. If you need to temporarily use Node 20, you can set the ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true environment variable. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
2 ▶ Run actions/setup-java@v4
15 Warning: setup-java v4 is deprecated and will no longer receive updates. Please migrate to actions/setup-java@v5.
16 ▶ Installed distributions
29 Creating settings.xml with server-id: github
30 Writing to /home/runner/.m2/settings.xml
31 Cache hit for: setup-java-linux-x64-maven-7b144edc313641f57af2a9b8d351ae1a568c5481808b046f8a5b0879c86d798
32 (node:1173) [DEP0169] DeprecationWarning: url.parse() behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for url.parse()' vulnerabilities.
33 Received 17152474 of 17152474 (100.0%), 17.6 MB/sec
34 Cache Size: -16 MB (17152474 B)
35 /usr/bin/tar -xzf /home/runner/work/_temp/80c90174-2ace-4c8c-8892-be349a0eb184/cache.tgz -P -C /home/runner/work/agrosmart-monitor/agrosmart-monitor --use-compress-program unxzstd
36 Cache restored successfully
37 Cache restored from key: setup-java-linux-x64-maven-7b144edc313641f57af2a9b8d351ae1a568c5481808b046f8a5b0879c86d798
  
```

Figura 15. Configuración de Java 17 y restauración de la caché de Maven.

Lectura de la evidencia: el runner descargó el repositorio y quedó asociado al commit correctivo. Luego preparó Java 17 y recuperó la caché de Maven. Las advertencias de deprecación no son errores y no impidieron la ejecución.

8.2.3 Pruebas automatizadas exitosas

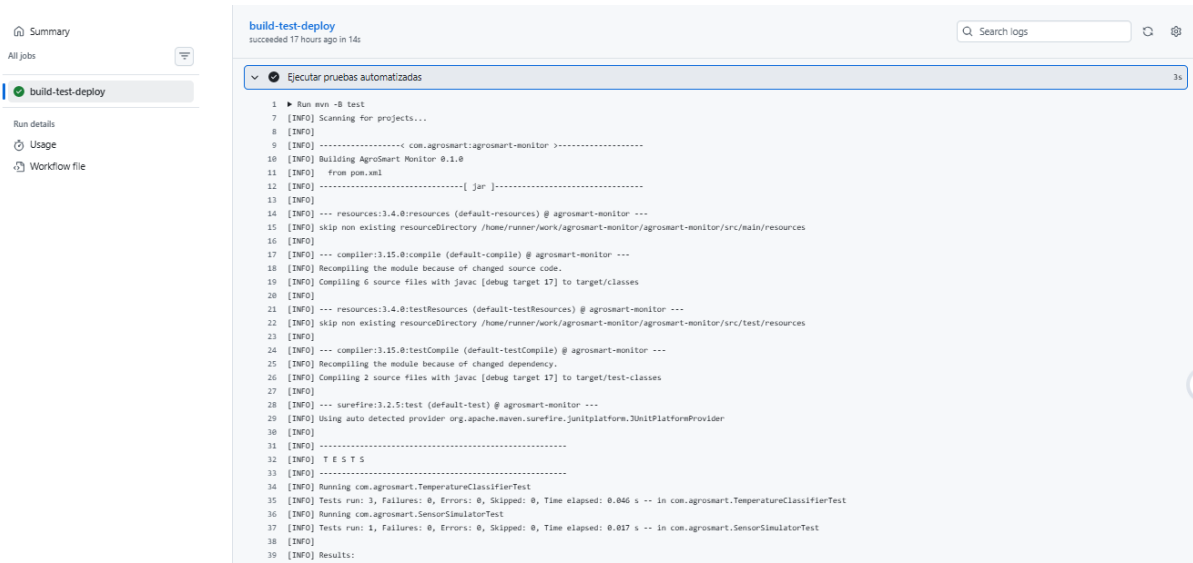


Figura 16. Inicio de las pruebas automatizadas con Maven y JUnit.

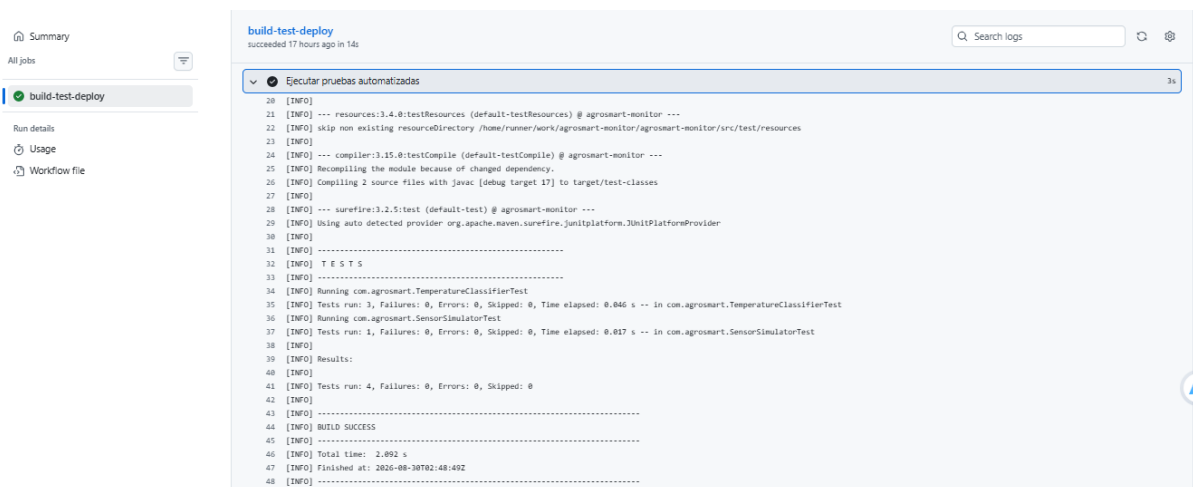
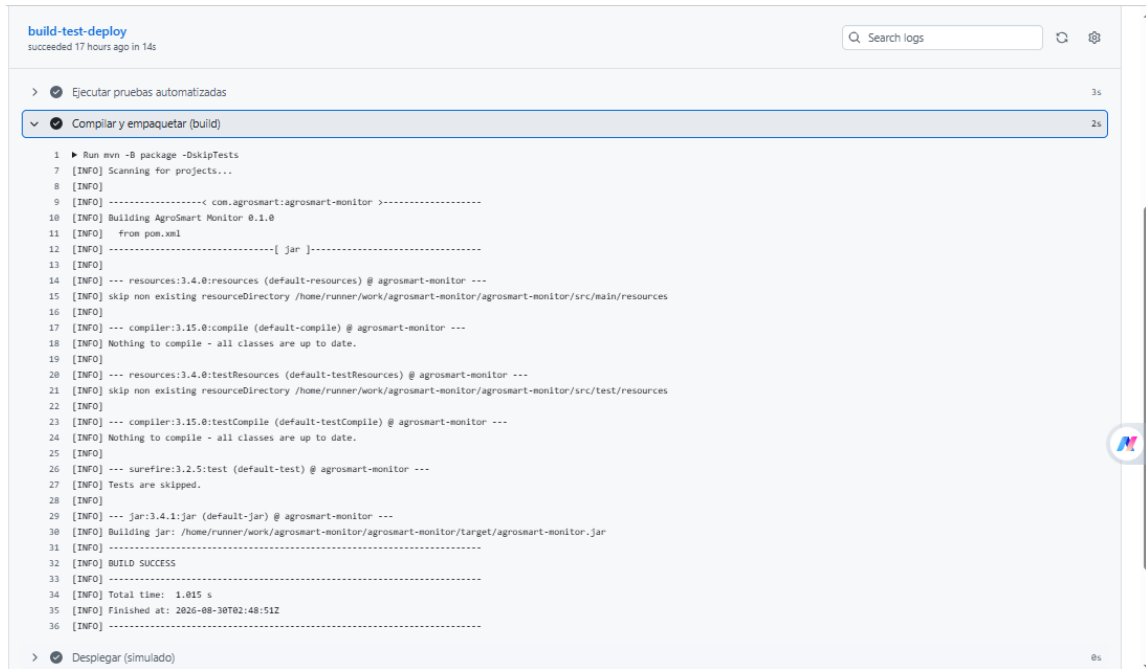


Figura 17. Resultado de pruebas: 4 ejecutadas, 0 fallos, 0 errores y BUILD SUCCESS.

Resultado de validación: TemperatureClassifierTest ejecutó tres pruebas y SensorSimulatorTest una. El resumen final registra Tests run: 4, Failures: 0, Errors: 0, Skipped: 0 y BUILD SUCCESS. La corrección quedó validada automáticamente.

8.2.4 Compilación, empaquetado y despliegue simulado

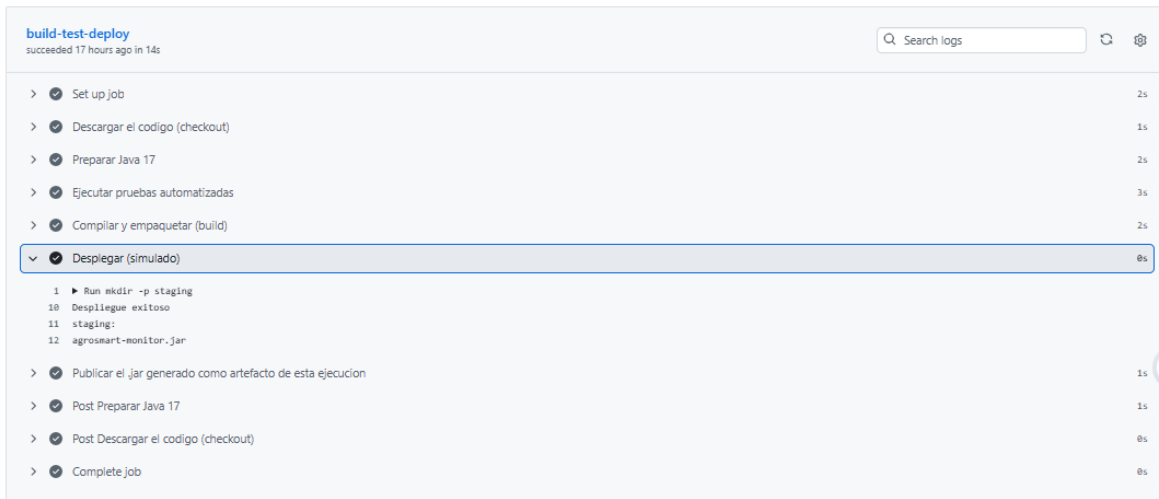


```

build-test-deploy
succeeded 17 hours ago in 14s

> Ejecutar pruebas automatizadas 3s
  > Compilar y empaquetar (build) 2s
    1 ▶ Run mvn -B package -DskipTests
    7 [INFO] Scanning for projects...
    8 [INFO]
    9 [INFO] ----- com.agrosmart:agrosmart-monitor -----
   10 [INFO] Building Agrosmart Monitor 0.1.0
   11 [INFO] from pom.xml
   12 [INFO] -----[ jar ]-----
   13 [INFO]
   14 [INFO] --- resources:3.4.0:resources (default-resources) @ agrosmart-monitor ---
   15 [INFO] skip non existing resourceDirectory /home/runner/work/agrosmart-monitor/agrosmart-monitor/src/main/resources
   16 [INFO]
   17 [INFO] --- compiler:3.15.0:compile (default-compile) @ agrosmart-monitor ---
   18 [INFO] Nothing to compile - all classes are up to date.
   19 [INFO]
   20 [INFO] --- resources:3.4.0:testResources (default-testResources) @ agrosmart-monitor ---
   21 [INFO] skip non existing resourceDirectory /home/runner/work/agrosmart-monitor/agrosmart-monitor/src/test/resources
   22 [INFO]
   23 [INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ agrosmart-monitor ---
   24 [INFO] Nothing to compile - all classes are up to date.
   25 [INFO]
   26 [INFO] --- surefire:3.2.5:test (default-test) @ agrosmart-monitor ---
   27 [INFO] Tests are skipped.
   28 [INFO]
   29 [INFO] --- jar:3.4.1:jar (default-jar) @ agrosmart-monitor ---
   30 [INFO] Building jar: /home/runner/work/agrosmart-monitor/agrosmart-monitor/target/agrosmart-monitor.jar
   31 [INFO] -----
   32 [INFO] BUILD SUCCESS
   33 [INFO] -----
   34 [INFO] Total time: 1.015 s
   35 [INFO] Finished at: 2026-08-30T02:48:51Z
   36 [INFO] -----
  > Desplegar (simulado) 0s
  
```

Figura 18. Compilación y generación del archivo target/agrosmart-monitor.jar.



```

build-test-deploy
succeeded 17 hours ago in 14s

> Set up job 2s
> Descargar el código (checkout) 1s
> Preparar Java 17 2s
> Ejecutar pruebas automatizadas 3s
> Compilar y empaquetar (build) 2s
  > Desplegar (simulado) 0s
    1 ▶ Run mkdir -p staging
    8 Despliegue exitoso
    9 staging:
   10 agrosmart-monitor.jar
   11
  > Publicar el jar generado como artefacto de esta ejecución 1s
  > Post Preparar Java 17 1s
  > Post Descargar el código (checkout) 0s
  > Complete job 0s
  
```

Figura 19. Despliegue simulado del archivo .jar en la carpeta staging.

Efecto del éxito: como las pruebas pasaron, el pipeline habilitó las etapas que antes habían sido omitidas. Maven generó el .jar y el paso Desplegar (simulado) verificó su presencia en staging.

8.2.5 Publicación del artefacto



Figura 20. Publicación satisfactoria del archivo .jar como artefacto de la ejecución.

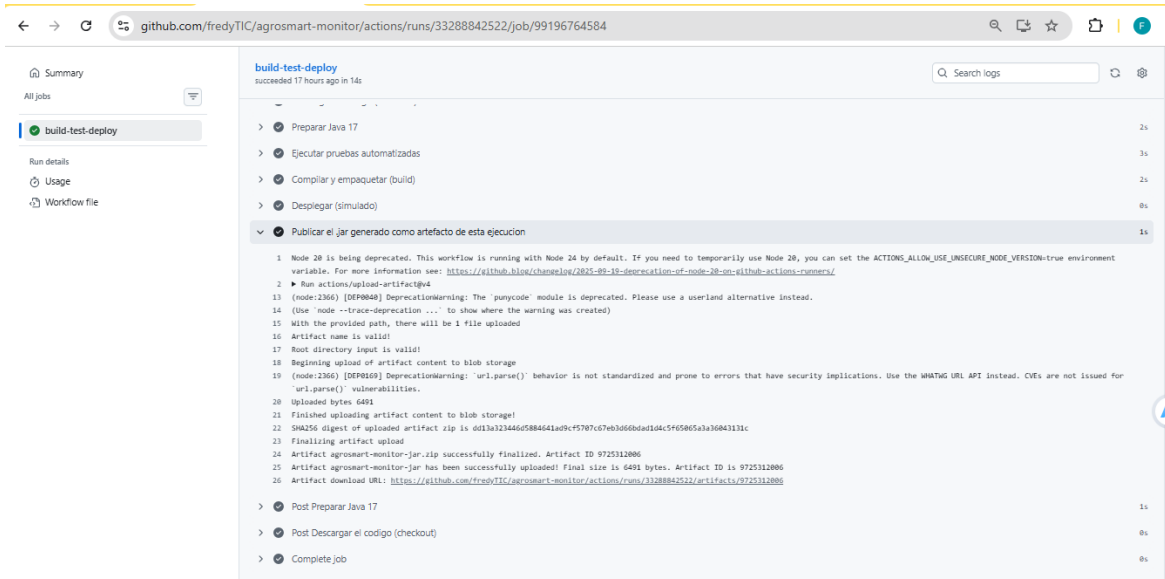


Figura 21. Confirmación del artefacto: 6491 bytes e identificador 9725312006.

Evidencia de entrega: GitHub Actions cargó agrosmart-monitor.jar, finalizó el artefacto y generó una URL de descarga. Esta evidencia demuestra que el pipeline no solo compiló, sino que produjo una salida recuperable asociada a la ejecución #8.

8.2.6 Limpieza y cierre del job exitoso

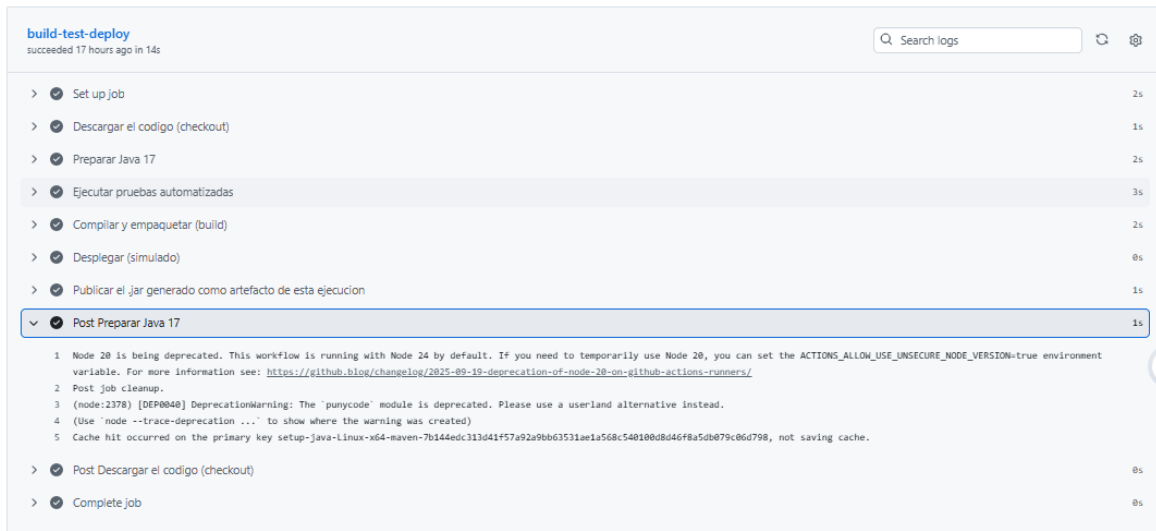


Figura 22. Limpieza posterior de la configuración de Java 17.

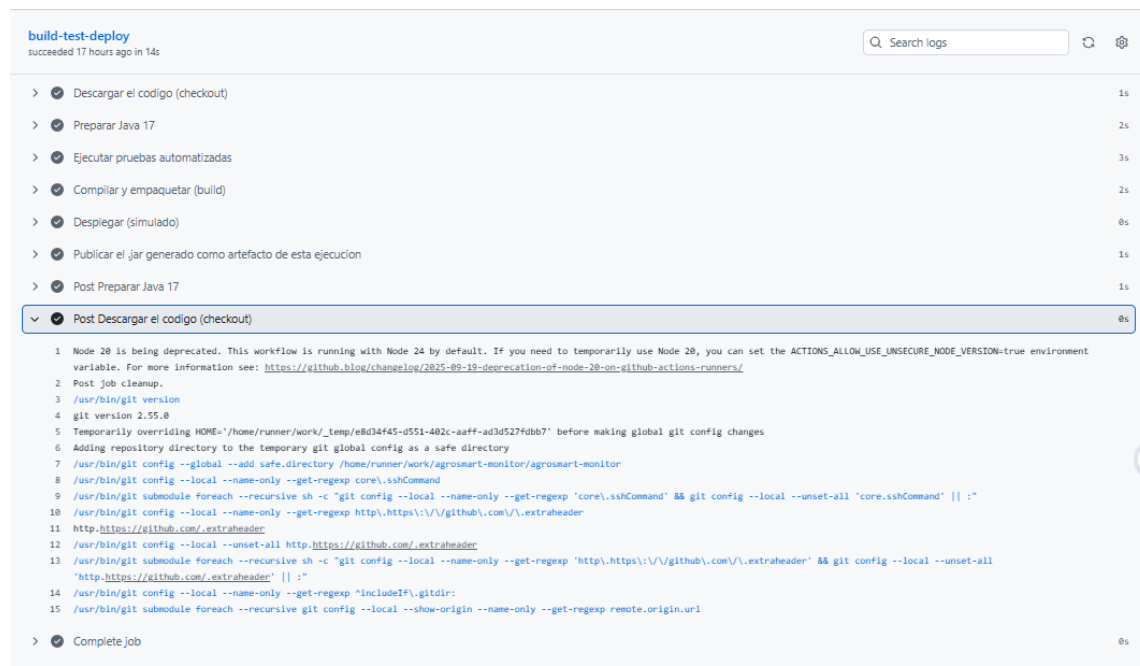


Figura 23. Limpieza de la configuración temporal utilizada durante el checkout.

Interpretación: los pasos Post Preparar Java 17 y Post Descargar el código son tareas automáticas de cierre. También aparecen en ejecuciones fallidas; por ello, su presencia debe interpretarse junto con el estado general del job.

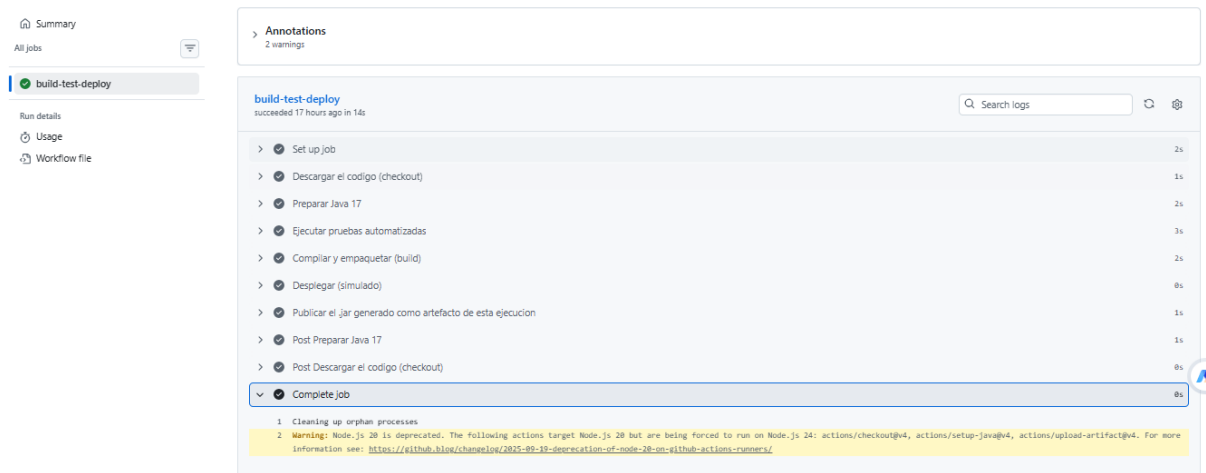


Figura 24. Complete job: cierre del runner después de una ejecución satisfactoria.

Cierre de la ejecución: Complete job libera los procesos del runner. La advertencia sobre Node.js 20 recomienda actualizar las versiones de algunas acciones, pero la ejecución se mantiene en Success porque no se produjo ningún error.

8.3 Resultados observados en la ejecución #8

Dato	Valor	Interpretación
Estado general	Success	La ejecución correctiva terminó satisfactoriamente.
Pruebas	4 / 0 / 0 / 0	Cuatro pruebas, cero fallos, cero errores y cero omisiones.
Build	BUILD SUCCESS	Maven generó target/agrosmart-monitor.jar.
Despliegue	Ejecutado	El .jar fue copiado a staging.
Artefacto	6491 bytes	El archivo fue publicado con ID 9725312006.

9. Cuadro comparativo de la trazabilidad

El cuadro siguiente presenta en paralelo lo observado en los logs de la ejecución fallida #7 y de la ejecución exitosa #8. La comparación muestra cómo una corrección puntual modifica el recorrido posterior del mismo pipeline.

Aspecto trazable	Ejecución fallida #7	Ejecución exitosa #8
Cambio asociado	La prueba esperaba NORMAL para 10.1 °C.	La prueba fue corregida para esperar ALERT.
Estado	Failed; el job se detuvo.	Success; el flujo llegó hasta Complete job.
Anotaciones	1 error y 2 advertencias.	2 advertencias y ningún error.
Pruebas	4 ejecutadas; 1 fallo.	4 ejecutadas; 0 fallos.
Log clave	expected NORMAL but was ALERT / BUILD FAILURE.	Failures: 0 / Errors: 0 / BUILD SUCCESS.
Build	Omitido por exit code 1.	Ejecutado; se generó agrosmart-monitor.jar.
Deploy simulado	Omitido.	Ejecutado; el .jar se copió a staging.
Artefacto	No se publicó.	Publicado; 6491 bytes, ID 9725312006.
Pasos Post	Se ejecutaron para limpiar el runner.	También se ejecutaron para limpiar el runner.
Efecto DevOps	Bloqueó una entrega inconsistente.	Permitió entregar el artefacto después de validar la corrección.

Conclusión comparativa: la trazabilidad no consiste únicamente en observar un ícono rojo o verde. Permite seguir la cadena cambio → commit → ejecución → job → paso → log → impacto. Al corregir la prueba, esa misma cadena pasó de bloquear build, deploy y artefacto a completar todas las etapas.

10. Resultados obtenidos

- Se automatizó la validación del proyecto ante cambios en la rama main.
- Las pruebas protegieron la regla de negocio antes del empaquetado.
- Los logs permitieron localizar la clase, el método, la línea y la diferencia entre el valor esperado y el real.
- Las etapas de build, despliegue y publicación se bloquearon automáticamente después del fallo.
- El escenario demostró la utilidad de la retroalimentación rápida y la trazabilidad en DevOps.
- La ejecución correctiva validó las cuatro pruebas y completó el build, el despliegue simulado y la publicación del artefacto.
- La comparación entre ambas ejecuciones permitió verificar la trazabilidad completa desde el cambio hasta el efecto producido en cada etapa del pipeline.

11. Conclusiones

- AgroSmart Monitor aplicó prácticas iniciales de DevOps al conectar el repositorio con pruebas, construcción y entrega automatizadas.
- El job build-test-deploy permitió ejecutar en orden los pasos del pipeline y controlar que ninguna etapa posterior continuara después de una prueba fallida.
- La trazabilidad no se limita a observar un ícono rojo o verde: exige relacionar el commit con la ejecución, el job, el paso, el log, la causa y el impacto.
- La detección expected NORMAL but was ALERT evidenció que el pipeline puede impedir la entrega de un cambio que contradice la regla de negocio.
- La transición de Failed a Success demuestra el ciclo de retroalimentación DevOps: detectar, diagnosticar, corregir, volver a ejecutar y entregar una salida validada.

12. Evidencias mínimas que acompañan la entrega

- ☐ URL del repositorio GitHub del equipo.
- ☐ Archivo del workflow incluido en .github/workflows/.
- ☐ Historial de commits.
- ☐ Captura de una ejecución fallida y explicación de la causa.
- ☐ Captura de la ejecución exitosa posterior a la corrección.
- ☐ README con instrucciones para ejecutar el proyecto y comprender el pipeline.